

[v1.0]项目状况

1. 项目当前实现状态分析

根据你提供的代码库 (CourseSelectionSystem_knowledge_base.md 及源文件), 目前项目处于 **v1.0 原型阶段**, 实现了基础的 **内存版** 选课逻辑。

- **已实现功能** :
 - **架构基础** : 基于 C++23 Modules 的工程结构, 分出了 `domain` (领域层) 和 `app.controller` (应用层)。
 - **领域实体** :
 - `Student` : 包含 ID、姓名、选课列表。实现了 `enrollIn` (选课) 和 `dropCourse` (退课)。
 - `Course` : 包含 ID、名称、容量。实现了 `isFull` (容量检查) 和 `acceptEnrollment` (报名)。
 - **业务逻辑** : 实现了简单的 **容量限制** (`size < capacity`) 和 **重复选课检查 1**。
 - **运行模式** : `SystemController` 使用硬编码的 Mock 数据 (`m_students.push_back(new Student...)`) 进行测试, 没有交互式 CLI, 也没有数据库持久化。
- **缺失核心功能 (依据需求文档)** :
 - **技术架构** : 缺失 **数据管理层 (Infrastructure)** 和 **表现层 (Presentation)** 。
 - **持久化** : 缺失 PostgreSQL 数据库连接及 **代理者模式 (Proxy Pattern)** 实现。
 - **业务规则** : 缺失 **时间冲突检测** (Timeslot overlaps)。
 - **角色权限** : 缺失 教师 (Teacher) 和 教学秘书 (Secretary) 角色及其特定操作 (如录入成绩、排课) 。
 - **交互** : 缺失 CLI 菜单系统 6。

2. 下一步分工规划 (Lead vs Member)

为了确保作为 **主负责人 (Lead)** 的你掌握核心技术并拿高分, 建议你负责 **架构核心、难点算法与数据库底层**, 而将 **重复性高、逻辑简单** 的 UI 和部分业务逻辑分配给成员。

分工策略表

模块/层级	任务内容	负责人	难度	理由 (基于评分关键)
Infrastructure (数据管理层)	PostgreSQL 代理模式实现 1. 封装 <code>libpqxx</code> 连接池 2. 实现 <code>StudentProxy</code> / <code>CourseProxy</code> 3. 实现对象-关系映射 (ORM) 逻辑	高扬	☆☆☆☆☆	评分关键：文档明确要求“代理者模式+PostgreSQL”，且领域层严禁出现 SQL。这是最核心的技术约束 7777。+1
Domain (领域层核心)	时间冲突检测算法 1. 定义 <code>Timeslot</code> 类 2. 实现 <code>bool overlaps()</code> 算法	高扬	☆☆☆☆	涉及核心业务逻辑算法，是文档要求的“关键技术演进”之一 8888。+1
Presentation (表现层)	CLI 菜单系统 1. <code>LoginMenu</code> (登录) 2. <code>StudentMenu</code> / <code>TeacherMenu</code> (循环菜单) 3. 处理用户输入与输出格式化	张涛	☆☆	代码量大但逻辑简单 (cin/cout/switch)，属于体力活，适合成员完成 9。
Database (数据库设计)	SQL 建表脚本 编写 <code>schema.sql</code> (Users, Courses, Enrollments 表)	张涛	☆	对照文档抄写 SQL 语句即可，难度低 10。
Domain (领域层扩展)	教师与成绩管理 1. 新增 <code>Teacher</code> 类 2. 实现 <code>grade()</code> (打分) 逻辑 3. 简单的日志输出	张涛	☆☆	逻辑相对独立且简单 (setter/getter)，不影响核心架构 11。
Application (应用逻辑层)	Controller 整合 将 Proxy 和 Domain 组装，协调各层调用	高扬	☆☆☆	需要理解全系统架构，确保“应用层不直接访问数据库”的约束 12。

3. 开发路线图 (Roadmap)

建议按照 **自底向上 (Bottom-Up)** 的顺序开发，最后在 Controller 汇合。

Phase 1: 基础设施搭建 (本周重点)

- Member:** 根据需求文档 4.1 章节，编写 PostgreSQL 的 `create_tables.sql` 并建库 13。
- Lead:** 配置 `libpqxx` 环境，创建 `infrastructure` 模块，编写 `PostgresConnection` 单例，打通 C++ 到数据库的连接。

Phase 2: 核心功能并行开发

1. Lead (Proxy模式):

- 实现 `StudentProxy::findById()` : 编写 SQL JOIN 查询, 将数据库数据转换为 `Student` 对象 14。
- 实现 `StudentProxy::save()` : 将内存对象状态写回数据库。

2. Lead (冲突检测):

- 修改 `Course` 类, 加入 `Timeslot` 属性。
- 实现 `Timeslot::overlaps`。

3. Member (UI与辅助业务):

- 实现 `Teacher` 类和成绩录入接口。
- 编写 `main.cpp` 中的 `LoginMenu`, 实现用户输入用户名/密码的流程 (暂时 mock 验证)。

Phase 3: 集成与重构

1. Lead: 重构 `SystemController`。

- 删除 `initialize()` 中的 mock 数据。
- 替换 `m_students` 内存向量为调用 `StudentProxy`。
- 示例代码逻辑:

```
1 // 你的 SystemController 将变成这样
2 void SystemController::performEnrollment(string sid, string cid) {
3     // 1. 使用 Proxy 从数据库加载 (Lead 工作)
4     Student student = _studentProxy.findById(sid);
5     Course course = _courseProxy.findById(cid);
6
7     // 2. 领域层业务判断 (Lead 工作)
8     if (student.canEnroll(course)) { // 包含冲突检测
9         student.enrollIn(course);
10        _studentProxy.save(student); // 持久化
11    }
12 }
```

2. Member: 将 CLI 菜单中的选项绑定到 `SystemController` 的方法上。

4. 给 Member 的任务派发单 (Template)

你可以直接把下面这段发给你的队友:

任务分配: `CourseSelectionSystem` - 阶段一

目标: 完成表现层界面和数据库基础。

你的任务 (Member):

1. 数据库准备:

- 查看《需求规格说明书》第 3 页 "4.1 核心表结构"。
- 在本地 PostgreSQL 中创建 `users`, `courses`, `enrollments` 三张表, 并录入一些测试数据。

2. CLI 界面开发 (src/presentation/):

- 创建 `presentation` 模块分区。
 - 实现 `showLoginMenu()`：接收用户输入 ID 和 密码。
 - 实现 `showStudentMenu()`：显示“1. 选课, 2. 退课, 3. 查课表”等选项，使用 `std::cin` 获取用户选择。
 - 注意：目前不需要写具体功能，只需要把界面打印出来，并返回用户的选择即可。
3. 补充领域对象：
- 参考现有的 `Student` 类，新建 `Teacher` 类，添加 `assignGrade(courseId, studentId, score)` 方法的空实现。

你 (Lead) 现在应该立刻开始做的是：研究 `libpqxx` 库的引入，并开始写 `StudentProxy` 的头文件设计。

高扬的分工（开发任务单：CourseSelectionSystem）

本阶段你的核心任务是打通 数据管理层 (Infrastructure) 与 领域层 (Domain) 的核心算法。你是系统的基石，你的代码质量直接决定了后续集成的难度。

- 技术栈: C++23 (Modules), `libpqxx` (PostgreSQL Driver), CMake 4.1.1+
- 开发分支: `feature/infra-core`

任务清单 (Checklist)

1. 基础设施层搭建 (Infrastructure Layer)

目标: 建立与 PostgreSQL 的连接，并实现“代理者模式”的基础设施，确保上层业务逻辑不需要触碰 SQL。

- 模块位置: `src/CourseSelectionSystem/infrastructure/`
- 关键约束: 所有 SQL 语句必须且只能出现在这一层。

1.1 数据库连接单例 (PostgresConnection)

我们需要一个单例来管理数据库连接，避免频繁创建连接的开销。

- 文件: `src/CourseSelectionSystem/infrastructure/infra.db_connection.cppm`

```
1  export module course_system:infrastructure.db_connection;
2  import std;
3  // import <pqxx/pqxx>; // 注意: 根据你的环境配置 libpqxx 的导入方式, 可能需要
   module map 或传统 include
4
5  export class PostgresConnection {
6  public:
7      static PostgresConnection& instance() {
8          static PostgresConnection instance;
9          return instance;
10     }
11
12     // 获取原始连接对象 (供 Proxy 使用)
13     // pqxx::connection& get_connection() { return *m_conn; }
14
15     // 简单的执行接口, 用于测试连接
16     void testConnection() {
17         std::print("[Infrastructure] DB Connection Active.\n");
18     }
19
20 private:
21     PostgresConnection() {
22         // TODO: 读取配置文件或硬编码连接字符串
23         // m_conn = std::make_unique<pqxx::connection>("dbname=course
   user=postgres password=root host=localhost");
24         std::print("[Infrastructure] Connected to PostgreSQL.\n");
25     }
26     // std::unique_ptr<pqxx::connection> m_conn;
```

1.2 实现学生代理 (StudentProxy)

这是评分的关键点。你需要实现对象与关系数据库之间的映射 (ORM)。

- **文件:** `src/CourseSelectionSystem/infrastructure/infra.student_proxy.cppm`
- **功能:**
 - `findById`: 接收 ID, 执行 `SELECT * FROM users WHERE user_id = ...`, 组装 `Student` 对象。
 - `save`: 接收 `Student` 对象, 执行 `INSERT` 或 `UPDATE`。

```

1  export module course_system:infrastructure.student_proxy;
2  import :domain.student;
3  import :infrastructure.db_connection;
4  import std;
5
6  export class StudentProxy {
7  public:
8      // 从数据库加载学生 (包含基础信息)
9      // 注意: 根据需求文档, 后续还需要 JOIN 查询加载已选课程
10     static Student* findById(std::string_view id) {
11         // 1. 获取连接
12         // auto& C = PostgresConnection::instance().get_connection();
13         // 2. 执行 SQL (示例伪代码)
14         // pqxx::work W(C);
15         // pqxx::result R = W.exec(std::format("SELECT name FROM users WHERE
16         user_id = '{}'", id));
17
18         // 3. 组装对象
19         // if (R.empty()) return nullptr;
20         // return new Student(std::string(id), R[0][0].c_str());
21
22         // Phase 1 Mock 实现:
23         std::print("[Proxy] Loading Student {} from DB...\n", id);
24         return new Student(std::string(id), "Mock DB Student");
25     }
26
27     // 持久化学生状态
28     static void save(const Student& s) {
29         std::print("[Proxy] Saving Student state to DB
30         (UPDATE/INSERT)...\n");
31         // 这里的逻辑: 拆解 Student 对象, 生成 SQL 语句写入 users 和 enrollments
32         // 表
33     }
34 };

```

2. 领域层核心算法 (Domain Core)

目标: 实现时间冲突检测逻辑。这是业务逻辑中最复杂的部分。

- **参考文档:** 《需求规格说明书》2.2.1 章节
- **关键规则:** 同一 weekday 且 timeslot 重叠即为冲突。

2.1 定义时间槽 (Timeslot)

- 文件: `src/CourseSelectionSystem/domain/dom.timeslot.cppm`
- 要求: 必须实现 `overlaps` 方法。

```
1 export module course_system:domain.timeslot;
2 import std;
3
4 export struct Timeslot {
5     int weekday; // 1-5 (Mon-Fri)
6     int slot_id; // 1-6 (1=1-2节, 2=3-4节, ...)
7
8     // 核心算法: 检查两个时间槽是否冲突
9     bool overlaps(const Timeslot& other) const {
10         // 规则: 如果星期相同 且 节次相同, 则冲突
11         return (weekday == other.weekday) && (slot_id == other.slot_id);
12     }
13
14     std::string toString() const {
15         return std::format("Day {} Slot {}", weekday, slot_id);
16     }
17 };
```

2.2 升级课程实体 (Course)

修改现有的 `dom.course.cppm`, 加入 `Timeslot` 属性。

```
1 // 在 Course 类中添加:
2 private:
3     Timeslot m_time;
4
5 public:
6     // 修改构造函数以接受时间信息
7     Course(std::string id, std::string name, int capacity, int day, int
8 slot)
9         : m_id(id), m_name(name), m_capacity(capacity), m_time{day, slot} {}
10
11     const Timeslot& getTime() const { return m_time; }
```

3. 应用层整合预研 (Application Refactoring)

目标: 准备将 Mock 数据替换为 Proxy 调用。

- 此步骤主要是代码重构的准备工作, 不需要写新文件, 但在心中要有数。

在下一阶段 (Phase 2), 你需要将 `SystemController::performEnrollment` 修改为:

```
1 // 伪代码预览
2 void SystemController::performEnrollment(string sid, string cid) {
3     // 1. 冲突检测 (Domain Service)
4     Student* s = StudentProxy::findById(sid);
5     Course* c = CourseProxy::findById(cid);
6
7     // 2. 核心校验
8     for (auto enrolledCourse : s->getCourses()) {
```

```
9         if (enrolledCourse->getTime().overlaps(c->getTime())) {
10             throw std::runtime_error("Time Conflict!");
11         }
12     }
13
14     // 3. 执行
15     s->enrollIn(c);
16     StudentProxy::save(*s); // 存回数据库
17 }
```

你的下一步行动 (Action Items)

1. **环境配置:** 确保本地开发环境安装了 `PostgreSQL` 和 `libpqxx` 开发库。
2. **创建文件:** 按照上述代码片段, 创建 `infrastructure` 模块和 `dom.timeslot` 模块。
3. **CMake 配置:** 更新 `src/CourseSelectionSystem/CMakeLists.txt`, 链接 `libpqxx` 库 (通常需要 `find_package(libpqxx REQUIRED)` 和 `target_link_libraries(... libpqxx::pqxx)`).
4. **单元测试:** 在 `main.cpp` 中编写一段测试代码, 实例化 `Timeslot` 并测试 `overlaps` 函数是否按预期工作。

完成这些后, 你将拥有一个可以连接数据库并进行核心业务计算的后端原型, 等待组员的 UI 接入即可。

张涛的分工（开发任务单: CourseSelectionSystem）

根据项目分工，第一阶段你主要负责 基础设施准备 (Database) 和 表现层 (Presentation/UI) 的搭建。请严格遵守以下技术栈和代码规范。

- 项目架构: 4层架构 (Presentation -> Application -> Domain -> Infrastructure)
- 语言标准: C++23 (必须使用 Modules)
- 构建系统: CMake 4.1.1+
- 分支: 请在 `feature/ui-db-init` 分支上开发

任务清单 (Checklist)

1. 数据库初始化 (Infrastructure Prep)

目标: 在 PostgreSQL 中建立符合需求文档的核心表结构。

- 参考文档: 《需求规格说明书》4.1 章节 (Page 4)
- 输出文件: `sql/init_schema.sql`

执行细节:

请编写 SQL 脚本创建以下三张表，并录入少量测试数据：

1. **users:** 包含 `user_id`, `name`, `password`, `role` (check约束: student/teacher/secretary)。
2. **courses:** 包含 `course_id`, `name`, `credit`, `capacity`, `teacher_id` (FK), `weekday`, `timeslot`。
3. **enrollments:** 包含 `student_id` (FK), `course_id` (FK), `score` (nullable)。

注意: 确保 `weekday` (1-5) 和 `timeslot` (1-6) 字段类型为整型，用于后续冲突检测。

2. 表现层搭建 (Presentation Layer)

目标: 实现基于控制台的交互菜单，暂不连接真实业务逻辑，仅做界面跳转。

- 模块位置: `src/CourseSelectionSystem/presentation/`
- 模块名称: `course_system:presentation`

2.1 创建模块接口文件

在 `src/CourseSelectionSystem/presentation/pres.module.cppm` 中定义模块：

```
1  /**
2   * @file      src/CourseSelectionSystem/presentation/pres.module.cppm
3   * @brief     Presentation layer partition
4   */
5   export module course_system:presentation;
6
7   import std;
8
9   export class UserInterface {
10  public:
```

```

11 // 显示主登录菜单, 返回登录成功的用户ID (模拟)
12 std::string showLoginMenu();
13
14 // 显示学生主菜单
15 void showStudentMenu(std::string_view studentId);
16
17 // 显示教师主菜单
18 void showTeacherMenu(std::string_view teacherId);
19 };

```

2.2 实现菜单逻辑 (UserInterface.cpp)

请实现上述接口。使用 `std::print` 和 `std::cin`。

- **showLoginMenu:** 提示输入 User ID 和 Password。目前不需要真实验证, 只要 ID 非空即返回 ID。
- **showStudentMenu:** 循环打印选项 :
 1. Enroll in Course (选课)
 2. Drop Course (退课)
 3. View Schedule (查课表)
 4. Logout (退出)
 - 暂时在控制台打印 "Feature [X] selected" 即可, 无需调用 Controller。

3. 领域层扩展 (Domain Extension)

目标: 补充 `Teacher` 实体类, 用于后续成绩管理。

- **文件位置:** `src/CourseSelectionSystem/domain/dom.teacher.cppm`

代码脚手架:

```

1  /**
2   * @file    src/CourseSelectionSystem/domain/dom.teacher.cppm
3   * @brief   Domain partition: Teacher entity
4   */
5   export module course_system:domain.teacher;
6
7   import std;
8
9   export class Teacher {
10  public:
11      Teacher(std::string id, std::string name) : m_id(id), m_name(name) {}
12
13      // 模拟录入成绩的操作
14      // return: true if success
15      bool assignGrade(std::string_view studentId, std::string_view courseId,
16      int score) {
17          // TODO: 真正的逻辑会在后续连接数据库后实现
18          // 目前仅打印日志
19          std::print("[Teacher {}] Assigned grade {} to Student {} for Course
20          {}\n",
21                      m_name, score, studentId, courseId);
22          return true;
23      }
24  };

```

```
23     std::string getId() const { return m_id; }
24
25 private:
26     std::string m_id;
27     std::string m_name;
28 };
```

提交要求 (Submission)

1. 文件结构确认:

Plaintext

```
1  sql/
2    └─ init_schema.sql
3  src/CourseSelectionSystem/
4    └─ presentation/
5        │   └─ pres.module.cppm (接口)
6        │   └─ UserInterface.cpp (实现)
7    └─ domain/
8        └─ dom.teacher.cppm
```

2. CMake 更新:

记得修改 src/CourseSelectionSystem/CMakeLists.txt，将新文件加入到 target sources 中。

3. 自测:

在 main.cpp 中临时调用 UserInterface ui; ui.showLoginMenu(); 确保菜单能跑通。

完成以上内容后，发起 **Pull Request**，我会进行 Code Review 并将其合并到主分支

核心分工对比 (一图看懂)

角色	高扬的职责	张涛的职责
定位	造发动机的人 (Core)	造车壳和内饰的人 (UI/DB)
负责层级	● 底层 + 核心层 (Infrastructure + Domain)	● 顶层 + 基础层 (Presentation + Database)
关键任务	1. 数据库连接与映射 (最难的 Proxy 模式) 2. 冲突检测算法 (解决上课时间重叠) 3. 系统总控 (重构 Controller)	1. SQL建表 (按文档写 create table) 2. 菜单界面 (写 cin/cout 交互) 3. 辅助功能 (教师打分接口)
涉及技术	C++ Modules, libpqxx, 复杂算法	SQL, C++ IO, 基础类设计

2. 协作流水线 (谁等谁?)

我们要像流水线一样配合，关键在于 Step 2 和 Step 3 的交接：

- Step 1: 各自开工 (并行)
 - ● 张涛：先去写 SQL 脚本把数据库表建好，同时把 CLI 菜单的“壳子”写出来（能显示选项，能输入，但暂时没反应）。
 - ● 高扬：搭建 libpqxx 环境，编写数据库连接池代码。
- Step 2: 关键卡点 (他等你)
 - ● 高扬：完成核心的 StudentProxy (从数据库读数据) 和 Timeslot (防冲突算法)。
 - 注：此时他的菜单是“空转”的，必须等高扬把这两个后端逻辑写好。
- Step 3: 最终合流 (你接他)
 - ● 高扬：把你的后端逻辑注入到 systemController 中。
 - ● 张涛：把你提供的接口（如 enroll()）绑定到他写的菜单按钮上，系统正式跑通。

一句话给组员的指令：

"你先把数据库表 (SQL) 建好，然后把控制台菜单 (UI) 的输入输出写好。等我把后端的数据库连接和算法搞定后，我们把你的界面和我的逻辑对接起来。"